# Midnight Multi-Party Wallet: Operations & Security Architecture

A guide for operating a privacy-preserving multi-party (M-of-N) wallet on Midnight, covering the transaction lifecycle per signer, key management, custodial HSM/TEE architecture, and coordination patterns.

This document focuses on the **operational perspective** -- what happens at each phase of a contract call, what keys are involved, and how to secure them in a custodial environment. The specific contract design (approval logic, state structure, circuit definitions) is owned by the contract team. For the general Midnight transaction lifecycle, see TRANSACTION_LIFECYCLE.md.

---

## Table of Contents


- Transaction Lifecycle Per Call
- Private State Management
  - What is Private State?
  - The PrivateStateProvider Interface
  - Witness Functions: Bridging Private State into Circuits
  - Secret Key Flow Through the Transaction Lifecycle
  - Private State Lifecycle
- Cryptographic Keys Per Signer
- Custodial Security Architecture
  - Security Zones by Phase
  - Independent TEE Per Signer
  - Key Derivation Flow
  - TEE vs HSM Decision
- Coordination Service
- Cloud Deployment
- Comparison: Single-Signer vs Multi-Party


---

## Transaction Lifecycle Per Call

Every contract call (submit action, approve, execute, etc.) follows the same 9-phase lifecycle. In a multi-party wallet, each signer goes through this independently for their own actions.

| Phase | What Happens | Keys Used | Where |
|---|---|---|---|
| 1 | Load private state (signer secrets, authorization data needed by circuit witnesses) | Implementation-dependent (e.g., decryption keys for encrypted storage) | Signer's storage |
| 2 | Circuit execution (CPU-only)<br>- Contract-level authorization checks (e.g., set membership, signature verification)<br>- Contract logic | CoinPublicKey (32-byte hash) | Signer's machine |
| 3 | ZK proof generation (contract circuit only)<br>- Proves contract authorization<br>- Proves circuit correctness<br>- NO wallet/shielded keys used | CoinSecretKey (32-byte hash)<br>Contract-specific secrets (authorization data, paths) | Proof server |

```
 4     Balance transaction                 CoinSecretKey              Wallet
       - Add DUST inputs for fee payment    (32-byte hash)
       - Resolve token imbalances          EncryptionSecretKey
         (add shielded/unshielded           (Jubjub scalar)
          inputs/outputs if needed)        DustSecretKey
       - Compute nullifiers

4b     ZK proof generation                 CoinSecretKey              Wallet /
       (balancing/wallet circuit)           (32-byte hash)            Proof server
       - Proves shielded coin ownership    EncryptionSecretKey
       - Proves DUST fee payment            (Jubjub scalar)
                                           DustSecretKey

 5     Sign unshielded inputs              SigningKey (BIP-340,       Wallet / HSM
       - BIP-340 Schnorr signature           secp256k1)
       - Per unshielded token input
       - Only needed when tx uses
         unshielded tokens

 6     Finalize                            None                       Wallet
       - Attach signatures
       - Seal transaction (establish
         binding -- forget intermediate
         data, retain only commitments)

 7     Submit to node                      None                       Network
       - JSON-RPC to Midnight node
       - Transaction IDs known before
         submission (from binding data)

 8     Node validates                      None (node-side)           Midnight node
       - Verify ZK proofs
       - Verify BIP-340 signatures
         (if unshielded inputs present)
       - Apply ledger operations

 9     Confirmation                        None                       Signer's machine
       - Poll indexer for inclusion
       - Update local private state
```

### Phase 1: Load Private State

**Where:** Signer's machine only -- nothing leaves **Keys used:** Implementation-dependent (e.g., storage decryption keys)

The signer's private state is loaded from local storage. This includes any contract-specific secrets needed by the circuit witnesses (e.g., authorization secrets, set membership paths, private data with signatures). Whether additional keys are needed depends on the security measures implemented around the private state provider -- encrypted backends require decryption keys, TEE-sealed storage requires enclave attestation, etc.

| Environment | Backend | Encryption | Persistence |
|---|---|---|---|
| Browser (DApps) | IndexedDB | AES-256-GCM (same scheme as CLI) | Survives restarts; lost on browser cache wipe |
| Server (custodial) | TEE-protected storage | Encrypted at rest, sealed to enclave identity | Persistent |

| | | | |
|---|---|---|---|
| CLI | LevelDB on disk | AES-256-GCM (PBKDF2, 100k iterations) | Survives restarts |

### Phase 2: Circuit Execution (CPU-only, no ZK proofs)

**Where:** Signer's machine, in the Compact Runtime (compiled Compact -> JavaScript) **Keys used:** `CoinPublicKey` (32-byte hash) -- injected into `CircuitContext`

The compiled Compact circuit runs as pure JavaScript. No ZK proving happens here -- it's just executing the contract logic locally to produce an execution trace.

```
// submitCallTx() internally builds a CircuitContext:
const circuitContext = createCircuitContext(
  contractAddress,
  coinPublicKey,        // <-- 32-byte hash derived from CoinSecretKey
  currentContractState, // <-- fetched from indexer
  privateState,         // <-- from private state provider (Phase 1)
);

// Executes the compiled Compact circuit (pure JS, no ZK proving)
const result = contract.circuits.someAction(circuitContext, ...args);
```

**What happens inside the circuit:**

1. **Witness calls** -- the circuit calls witness functions to access the signer's private state (secrets, authorization data, signed attestations, etc.)
2. **`ownPublicKey()`** -- reads the signer's `CoinPublicKey` (32-byte hash) from the circuit context. This is the signer's on-chain identity
3. **Authorization checks** -- the contract verifies the signer is authorized (e.g., set membership proofs, signature verification, threshold checks)
4. **Contract logic** -- executes the action (update state, record approvals, etc.)
5. **Ledger writes** -- produces public transcript operations that will be applied on-chain. Only `disclose()` d values become visible

**Output:** An `UnprovenTransaction` containing:

| Field | Description |
|---|---|
| `publicTranscript` | Ledger operations -- visible on-chain after inclusion |
| `privateTranscriptOutputs` | Witness results (encrypted) -- never revealed |
| `partialProofData` | Circuit execution trace for the ZK prover (contains secret witness wires) |
| `input` / `output` | Circuit arguments and return value |

### Phase 3: ZK Proof Generation -- Contract Circuit (Proof Server)

**Where:** Proof server (localhost:6300 in dev, co-located in TEE in production) **Keys used:** `CoinSecretKey` (32-byte hash) + all contract-specific secrets **Protocol:** HTTP POST

This phase generates a ZK proof **only for the contract circuit** -- proving that the contract logic was executed correctly, without revealing any secret inputs. Wallet-held keys (shielded coin keys, encryption keys, DUST keys) are **not** involved in this proof; they are used later during the balancing proof (Phase 4b).

```
Signer's machine                        Proof Server (:6300)
     |                                       |
     |  POST /prove                          |
```

```
|  {                                    |
|    circuitId: "someAction",           |
|    partialProofData: {                |
|      CoinSecretKey (32-byte hash),    |  <-- Zswap protocol witness
|      contract-specific secrets,       |  <-- Contract witnesses
|      authorization paths/data,        |  <-- Contract witnesses
|      circuit execution trace,         |
|    }                                  |
|  }                                    |
|-------------------------------------->|
|                                       | 1. Loads proving key for circuit
|                                       |
|                                       | 2. Runs Plonk prover:
|                                       |    - CoinSecretKey is a private
|                                       |      witness wire (proves coin
|                                       |      ownership via Zswap protocol)
|                                       |    - Contract secrets are private
|                                       |      witness wires (prove
|                                       |      authorization)
|                                       |    - Generates zk-SNARK proof
|                                       |
|  { proof: Uint8Array }                |
|<--------------------------------------|
```

**Critical: Secret key exposure to proof server.** The `CoinSecretKey` (32-byte hash) is sent to the proof server as part of the witness data. The prover **must** have the raw key bytes -- it's an input wire to the arithmetic circuit. This is why the proof server must run inside the TEE in production.

**What the proof proves (without revealing the secrets):**

1. The signer owns the `CoinPublicKey` they claim (via `CoinSecretKey` witness)
2. The contract-level authorization checks passed (via contract-specific secret witnesses)
3. The circuit logic was executed correctly

**Output:** `Transaction<SignatureEnabled, Proof, PreBinding>` -- proven, but not yet balanced or signed.

### Phase 4: Balance the Transaction

**Where:** Wallet (Lace extension or WalletFacade SDK) **Keys used:** `CoinSecretKey` (32-byte hash) + `EncryptionSecretKey` (Jubjub scalar) + `DustSecretKey`

Balancing resolves two concerns: (1) adding **DUST inputs** to pay the transaction fee, and (2) resolving any **token imbalances** in the contract transaction (e.g., if the contract transfers shielded or unshielded tokens, the wallet must add matching inputs/outputs). Each signer pays their own fees from their own DUST balance.

**Via Lace (browser):**

```
import type { DAppConnectorAPI } from '@midnight-ntwrk/dapp-connector-api';

const serialized = tx.serialize();
const serializedStr = uint8ArrayToHex(serialized);
// Lace performs balance + sign + finalize internally
const result = await wallet.balanceUnsealedTransaction(serializedStr, {});
```

**Via WalletFacade (CLI/server):**

```
const recipe = await walletFacade.balanceUnboundTransaction(
  tx,
```

```
  { shieldedSecretKeys, dustSecretKey },
  { ttl }
);
```

**What balancing does:**

1. **Adds DUST inputs for fee payment** -- DUST is the fee token on Midnight; the wallet selects spendable DUST coins using the `DustSecretKey`
2. **Resolves token imbalances** -- if the contract transaction creates or consumes shielded/unshielded tokens, the wallet adds corresponding inputs or outputs to make the transaction balance
3. **Calculates the transaction fee** -- based on tx size x fee rate from cost model
4. **Computes nullifiers** -- uses `CoinSecretKey` to derive nullifiers for spent coins (prevents double-spending)
5. **Decrypts shielded coins** -- uses `EncryptionSecretKey` (Jubjub scalar) to identify and decrypt spendable shielded coins via ECDH

### Phase 4b: ZK Proof Generation -- Balancing/Wallet Circuit

**Where:** Wallet / Proof server **Keys used:** `CoinSecretKey` (32-byte hash) + `EncryptionSecretKey` (Jubjub scalar) + `DustSecretKey`

After balancing adds the necessary inputs/outputs, a **separate ZK proof** is generated for the wallet-related parts of the transaction -- proving ownership of shielded coins and DUST used for fee payment. This is distinct from the contract circuit proof in Phase 3.

**Key distinction:** The wallet's shielded coin keys ( `EncryptionSecretKey` , `DustSecretKey` ) are **never** exposed to the contract circuit proof (Phase 3). They only appear as witness wires in this wallet-level proof. This separation ensures that contract circuits cannot access wallet keys, and wallet proofs cannot access contract secrets.

### Phase 5: Sign Unshielded Inputs (BIP-340 Schnorr)

**Where:** Wallet (or HSM in custodial setups) **Keys used:** `SigningKey` (BIP-340, **secp256k1** -- a different curve from Jubjub)

BIP-340 signatures **authorize spends of unshielded tokens only** -- they do not authorize the transaction as a whole. If a transaction does not consume any unshielded tokens (e.g., a pure shielded transaction or a contract call that doesn't move unshielded tokens), no BIP-340 signatures are needed at all. Each unshielded input requires its own separate signature.

**Key derivation:**

```
HD Seed (24-word BIP39 mnemonic)
  -> BIP32 derivation path (NightExternal role)
    -> Raw private key bytes (32 bytes, secp256k1 scalar)
      -> signingKeyFromBip340(bytes)
        -> SigningKey (hex string with 3-byte BIP-340 version prefix)
```

**Via Lace:** Signing happens inside the extension, transparent to the DApp. User sees a confirmation popup.

**Via SDK:**

```
const signedRecipe = await walletFacade.signRecipe(
  recipe,
  (payload: Uint8Array) => ledger.signData(signingKey, payload)
  //                       ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  //                       BIP-340 Schnorr signature on secp256k1
  //                       Input: serialized unshielded input payload
  //                       Output: 64-byte Schnorr signature
);
```

**Via HSM (custodial):** The unshielded input payloads are sent to the HSM; only the 64-byte signatures come back. The `SigningKey` never leaves the HSM.

Each **unshielded token input** in the transaction gets a separate BIP-340 signature.

### Phase 6: Finalize

**Where:** Wallet **Keys used:** None (sealing step)

```
const finalizedTx = await walletFacade.finalizeRecipe(signedRecipe);
```

1. Attaches all BIP-340 signatures (if any) to the transaction
2. **Seals the transaction** -- intermediate data that would allow free modification of transaction contents is forgotten; only the binding commitments remain. This transitions the transaction from `PreBinding` to `Binding` state
3. Produces `FinalizedTransaction` -- ready for network submission

No secret material is needed. The finalized transaction contains only proofs, signatures, and binding commitments (all public).

### Phase 7: Submit to Node

**Where:** Signer's machine -> Midnight Node (JSON-RPC) **Keys used:** None

Transaction identifiers are **known before submission** -- they are derived from the binding data established during finalization. Each piece of binding data forms a separate identifier, accessible via `Transaction.prototype.identifiers()` (source).

```
// Transaction IDs are known before submission
const txIds = finalizedTx.identifiers();

// CLI / server
await walletFacade.submitTransaction(finalizedTx);

// UI (via Lace -- use @midnight-ntwrk/dapp-connector-api for proper typing)
import type { DAppConnectorAPI } from '@midnight-ntwrk/dapp-connector-api';
await wallet.submitTransaction(serializedStr, {});
```

The serialized `FinalizedTransaction` is sent to the Midnight node. No secrets are transmitted -- the transaction contains only proofs, signatures, and binding commitments.

### Phase 8: Node Validates & Includes in Block

**Where:** Midnight Node **Keys used:** None from the signer's perspective. The node uses on-chain verification data (circuit verifier blobs deployed with the contract) internally.

The node performs:

1. **Deserialization** -- parse the transaction bytes
2. **BIP-340 signature verification** (if unshielded inputs present) -- verifies the signer authorized each unshielded token spend. Uses the signer's public key (derived from `SigningKey`)
3. **ZK proof verification** -- verifies both the contract circuit proof and the balancing/wallet proof using the on-chain verification data. This confirms all contract-level authorization checks passed and all coin ownership claims are valid, without revealing any private inputs
4. **Guaranteed operations** -- applies the public transcript to the ledger (state updates, approvals, etc.)
5. **Fallible operations** -- may partially succeed without rejecting the entire transaction
6. **Block inclusion** -- transaction included in block with `blockHeight`

### Phase 9: Confirmation & Private State Update

**Where:** Signer's machine, polling the indexer **Keys used:** None

```
// submitCallTx internally polls the indexer:
const txData = await publicDataProvider.watchForTxData(txId);

// On success, updates private state for future circuit calls:
await privateStateProvider.set(privateStateId, nextPrivateState);
```

The signer's local private state is updated with the new state resulting from the circuit execution. This ensures the next call starts from the correct state.

---

### Two Layers of Authorization

Each call involves up to two independent cryptographic authorizations:

| Layer | Mechanism | What It Proves | Curve | Phase |
|---|---|---|---|---|
| Contract-level | ZK proof (authorization checks defined by contract) | "I am authorized to perform this action on this contract" | Hash-based (coin keys) / BLS12-381 | 3 (generated), 8 (verified) |
| Wallet-level | ZK proof (shielded coin ownership, DUST fees) | "I own the shielded coins and DUST used in this transaction" | Jubjub (encryption keys) / BLS12-381 | 4b (generated), 8 (verified) |
| Unshielded spend | BIP-340 Schnorr signature | "I authorize spending this unshielded token input" | secp256k1 | 5 (signed), 8 (verified) |

The ZK proofs are always required. BIP-340 signatures are only needed when the transaction spends unshielded tokens. They use different keys on different curves and serve different purposes.

### Cryptographic Primitives Used Across Phases

| Component | Type | Algorithm | Used In |
|---|---|---|---|
| ZK proof system | BLS12-381 curve | Plonk zk-SNARK | Phase 3, 4b (prove), Phase 8 (verify) |
| Coin keys (`CoinSecretKey`, `CoinPublicKey`) | 32-byte hashes | SHA-256 with domain separators (spec) | Phase 2 (identity), Phase 3 (witness), Phase 4 (nullifiers) |
| Encryption keys (`EncryptionSecretKey`, `EncryptionPublicKey`) | Jubjub scalar / point | ECDH on Jubjub curve | Phase 4 (decrypt shielded coins), Phase 4b (wallet proof) |
| Unshielded input signing | secp256k1 | BIP-340 Schnorr | Phase 5 (sign), Phase 8 (verify) |
| Hashing (inside circuits) | N/A | Poseidon (`transientHash`) or SHA-256 (`persistentHash`) | Phase 2 (circuit) |

### HSM Compatibility by Phase

| Phase | Key Needed | Type | Can Stay in HSM? | Why |
|---|---|---|---|---|

| | | | | |
|---|---|---|---|---|
| 2. Circuit | `CoinPublicKey` | 32-byte hash | N/A (not secret) | Public key, no HSM needed |
| 3. Contract Proof | `CoinSecretKey` + contract secrets | 32-byte hash + varies | **No** -- must be in memory | Prover needs raw bytes as arithmetic circuit witness wires |
| 4. Balance | `CoinSecretKey` + `EncryptionSecretKey` + `DustSecretKey` | Hash + Jubjub scalar | **No** -- must be in memory | Needed for nullifier computation and shielded coin decryption. No HSM supports Jubjub |
| 4b. Wallet Proof | `CoinSecretKey` + `EncryptionSecretKey` + `DustSecretKey` | Hash + Jubjub scalar | **No** -- must be in memory | Prover needs raw bytes for wallet circuit witness wires |
| 5. Sign | `SigningKey` | secp256k1 (BIP-340) | **Yes** | BIP-340 Schnorr on secp256k1. HSM receives payload, returns 64-byte signature. Key never leaves. Only needed for unshielded inputs |

---

## Private State Management

Private state is the core privacy mechanism in Midnight. It holds the signer's secret data -- authorization secrets, signatures, paths, and any contract-specific private inputs. This data never leaves the signer's machine in plaintext; it enters circuits through witness functions and becomes private witness wires in the ZK proof.

### What is Private State?

Private state is a **per-contract, per-signer** data structure stored locally. It contains everything the circuit needs that must remain secret:

| Content | Example | Purpose |
|---|---|---|
| Authorization secrets | Private keys, secret tokens, PINs | Prove signer identity inside the circuit |
| Signed attestations | Schnorr signatures from external parties | Prove off-chain data integrity |
| Set membership proofs | Merkle paths, indices | Prove inclusion in an authorized set |
| Private business data | Scores, amounts, identifiers | Contract inputs that must not be disclosed |
| Computed intermediaries | Hash preimages, reduction values | Helper data for ZK-friendly arithmetic |

Private state is **not** the same as the on-chain contract state (the "ledger"). The ledger is public and shared; private state is local and secret.

### The PrivateStateProvider Interface

The SDK manages private state through the `PrivateStateProvider` interface. Every signer needs one:

```
interface PrivateStateProvider<PSI extends string, PS> {
  // Scoping -- isolates private state per contract
  setContractAddress(address: ContractAddress): void;

  // CRUD operations on private state
  get(privateStateId: PSI): Promise<PS | null>;
  set(privateStateId: PSI, state: PS): Promise<void>;
  remove(privateStateId: PSI): Promise<void>;
```

```
  clear(): Promise<void>;

  // Contract maintenance authority key (per contract)
  // Used for contract governance: updating verifier keys, changing
  // maintenance authorities. NOT for transaction signing.
  setSigningKey(address: ContractAddress, signingKey: SigningKey): Promise<void>;
  getSigningKey(address: ContractAddress): Promise<SigningKey | null>;
  removeSigningKey(address: ContractAddress): Promise<void>;
  clearSigningKeys(): Promise<void>;

  // Export/import (backup, migration)
  exportPrivateStates(options?: ExportOptions): Promise<PrivateStateExport>;
  importPrivateStates(data: PrivateStateExport, options?: ImportOptions): Promise<ImportResult>;
}
```

**Contract address scoping:** `setContractAddress()` must be called before any `get` / `set` / `remove` operation. The SDK calls this automatically during `findDeployedContract()` and `submitCallTx()`. This ensures that private state for contract A cannot accidentally be read when operating on contract B -- critical in a multi-party setup where a signer may participate in multiple contracts.

**Internal storage key:** `${contractAddress}:${privateStateId}` -- the contract address acts as a namespace prefix.

**Contract maintenance authority key:** The provider also stores a `SigningKey` per contract address via `setSigningKey()` / `getSigningKey()`. Despite the method name, this is **not** the BIP-340 key used for signing unshielded token spends (that key lives in the wallet). Instead, this is a **contract maintenance authority key** -- used for privileged governance operations such as updating a circuit's verifier keys ( `VerifierKeyInsert` , `VerifierKeyRemove` ) or changing the contract's maintenance authorities ( `ReplaceAuthority` ). These are serious operations (e.g., a verifier key update changes what proofs the contract accepts), so the key is stored separately and generated on deploy. See ADR-0021: Contract Maintenance Authorities and ledger maintenance types.

### Storage Backends

| Environment | Backend | Encryption | Persistence | Maintenance Key Storage |
|---|---|---|---|---|
| Browser (DApps) | IndexedDB | AES-256-GCM (same scheme as CLI) | Survives restarts; lost on browser cache wipe | Same IndexedDB store |
| Server (custodial) | TEE-protected storage | Encrypted at rest, sealed to enclave identity | Persistent | Sealed within TEE |
| CLI | LevelDB on disk | AES-256-GCM (PBKDF2, 100k iterations) | Survives restarts | LevelDB `signing-keys` sublevel |

### Witness Functions: Bridging Private State into Circuits

Witness functions are the **only way** for private state to enter a circuit. They act as a controlled bridge between the signer's local storage and the ZK execution environment.

**In the Compact contract (declarative):**

```
// Declare a witness -- the circuit can call this to get private data
witness getAuthorizationData(): [SecretToken, MerklePath, Uint<16>];

export circuit submitAction(actionData: Bytes<32>): [] {
    // Call the witness to retrieve private state
    const [secret, path, index] = getAuthorizationData();
```

```
    // Use the secret data in authorization checks
    // (these become private witness wires in the ZK proof)
    assert(verifyMembership(secret, path, index), "Not authorized");

    // Continue with contract logic...
}
```

**In the TypeScript witness implementation (runtime):**

```
// The witness receives a WitnessContext containing:
//   - privateState: the signer's current private state
//   - ledger: read-only view of the on-chain contract state

export const witnesses = {
  getAuthorizationData: ({
    privateState,  // <-- loaded from PrivateStateProvider.get() in Phase 1
    ledger,        // <-- fetched from indexer (public, read-only)
  }: WitnessContext<Ledger, MyPrivateState>): [
    MyPrivateState,    // <-- updated private state (returned back to provider)
    [SecretToken, MerklePath, bigint],  // <-- values exposed to the circuit
  ] => [
    privateState,      // pass through unchanged (or modify if needed)
    [
      privateState.secretToken,
      privateState.merklePath,
      privateState.memberIndex,
    ],
  ],
};
```

**Key points about witness functions:**

1. **Return tuple encourages immutability:** Every witness returns `[nextPrivateState, circuitValues]`. The first element is the (possibly updated) private state; the second is the data exposed to the circuit. This API shape is intentional -- by requiring the witness to return the next state rather than mutating the input, it encourages an immutable data flow pattern
2. **Private state evolution:** Witnesses can produce a new private state before returning it (e.g., increment a nonce, mark a secret as used). The new state flows back to the provider. Most witnesses return the input `privateState` unchanged
3. **Read-only ledger access:** Witnesses can read the current on-chain state via the `ledger` parameter, but cannot modify it
4. **Multiple witnesses per circuit:** A circuit can call multiple witnesses, each accessing different parts of the private state
5. **Computation in witnesses:** Witnesses can perform arbitrary TypeScript computation (division, modular arithmetic, data transformation) to prepare values that would be expensive in ZK circuits

## Secret Key Flow Through the Transaction Lifecycle

This diagram traces how secret keys and private state flow through each phase:

```
Phase 1: Load Private State
+-----------------------------------------+
| PrivateStateProvider.get(privateStateId)  |
|                                         |
| Returns: {                              |
```

```
|   authorizationSecret: 0xabc...,          |
|   merklePath: [...],                      |
|   memberIndex: 3,                         |
|   attestationSignature: {...},            |
|   ...                                     |
| }                                         |
+-------------------+-----------------------+
                    |
                    v
Phase 2: Circuit Execution (witnesses called here)
+-----------------------------------------+
| CircuitContext contains:                |
|   - coinPublicKey (32-byte hash, not secret)|
|   - contractState (from indexer)        |
|   - privateState (from Phase 1)         |
|                                         |
| Circuit calls witness:                  |
|   getAuthData() -->                     |
|     WitnessContext { privateState, ledger }|
|     --> [nextPrivateState, [secret, path]]|
|                                         |
| Witness values become circuit variables:  |
|   secret  --> private variable (hidden)   |
|   path    --> private variable (hidden)   |
|   assert(verify(secret, path)) --> public |
|             (pass/fail only)            |
|                                         |
| Output: UnprovenTransaction {           |
|   partialProofData: {                   |
|     witnessWires: [                     |
|       CoinSecretKey (32-byte hash),     |  <-- injected by SDK
|       secret (from witness),            |  <-- from private state
|       path (from witness),              |  <-- from private state
|       ...all private circuit variables  |
|     ]                                   |
|   }                                     |
| }                                       |
+-------------------+-----------------------+
                    |
                    v
Phase 3: Proof Generation
+-----------------------------------------+
| Proof server receives partialProofData  |
|                                         |
| All witness values are now private      |
| witness WIRES in the arithmetic circuit:  |
|                                         |
|   CoinSecretKey --> wire #0 (Zswap)     |
|   secret        --> wire #1 (contract)  |
|   path          --> wire #2 (contract)  |
|                                         |
| The prover SEES all secret values       |
| (they are raw bytes in memory)          |
|                                         |
| Output: ZK proof that:                  |
|   - All authorization checks passed     |
```

```
|   - CoinPublicKey matches CoinSecretKey   |
|   - Circuit logic executed correctly       |
|   WITHOUT revealing any wire values       |
+-------------------+-----------------------+
                    |
                    v
Phase 4-5: Balance + Sign
  (CoinSecretKey for nullifiers,
   SigningKey for BIP-340 -- no private
   state involved)
                    |
                    v
Phase 9: Update Private State
+-----------------------------------------+
| On successful inclusion:                |
|                                         |
| PrivateStateProvider.set(               |
|   privateStateId,                       |
|   nextPrivateState  // from witness return|
| )                                       |
|                                         |
| The updated private state is persisted  |
| for the next circuit call               |
+-----------------------------------------+
```

### Private State Lifecycle

The complete lifecycle of private state across a single transaction:

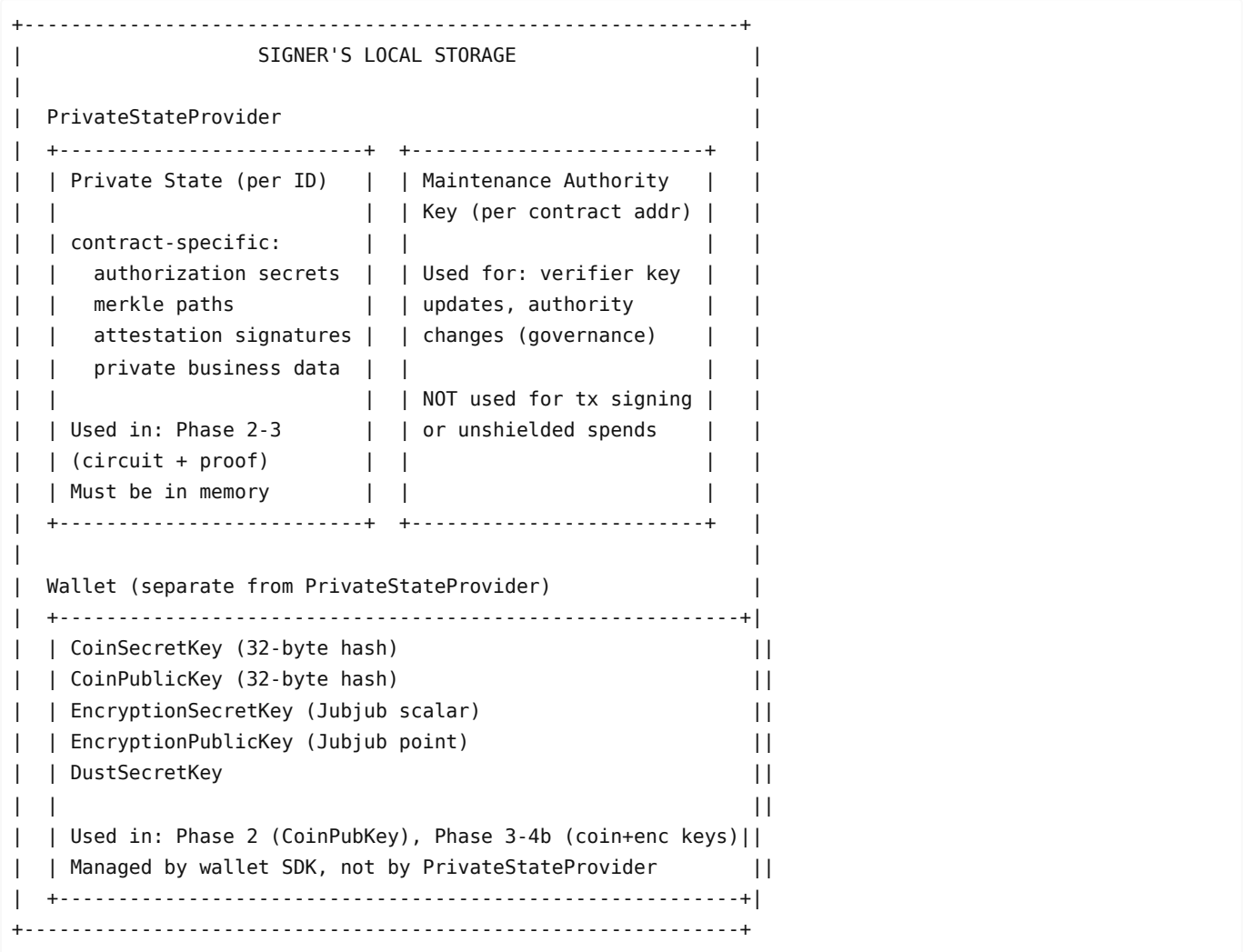
```
1. LOAD        PrivateStateProvider.get(id)
               Storage --> privateState object
                    |
2. INJECT      Circuit execution starts
               privateState --> WitnessContext
                    |
3. READ        Witness function reads privateState fields
               Returns [nextPrivateState, circuitValues]
                    |
4. EXECUTE     Circuit uses circuitValues as private variables
               Authorization checks run on secret data
               Circuit produces publicTranscript (on-chain effects)
                    |
5. ENCODE      SDK encodes all private variables into partialProofData
               (secret values become ZK witness wires)
                    |
6. PROVE       Proof server generates ZK proof
               Secret wires are consumed but NEVER revealed
                    |
7. SUBMIT      Transaction submitted (proof + signatures only)
               No private state data leaves the signer's machine
                    |
8. CONFIRM     Node validates proof and includes in block
                    |
9. PERSIST     PrivateStateProvider.set(id, nextPrivateState)
               nextPrivateState (from step 3) saved for next call
```

**State transitions during the lifecycle:**

- **Unchanged pass-through:** Most witnesses return `privateState` as-is. The same secrets are reused across multiple calls (e.g., an authorization secret that doesn't change)
- **Mutating witnesses:** Some witnesses modify private state before returning (e.g., incrementing a counter, marking a one-time secret as consumed). The updated state is only persisted if the transaction succeeds (Phase 9)
- **Rollback on failure:** If the transaction fails (Phase 8), the private state is **not** updated. The provider still holds the pre-transaction state, so the signer can retry

### Private State vs Protocol Keys

It is important to distinguish between private state (contract-level secrets) and protocol-level keys:




The `PrivateStateProvider` manages:

- **Contract private state** -- secrets needed by witness functions (authorization data, attestations, etc.)
- **Contract maintenance authority keys** -- one per contract address, for governance operations (verifier key updates, authority changes). Not related to transaction signing or unshielded spends

The **wallet** (Lace or WalletFacade) separately manages:

- **CoinSecretKey/CoinPublicKey** (32-byte hashes) -- for coin ownership proofs and nullifier computation
- **EncryptionSecretKey/EncryptionPublicKey** (Jubjub scalar/point) -- for shielded coin ECDH
- **DustSecretKey** -- for DUST fee payment
- **BIP-340 SigningKey** (secp256k1) -- for authorizing unshielded token spends (Phase 5)
- **HD seed** -- source of all derived keys

All three stores are needed for a complete transaction, but they serve different purposes and may have different security boundaries (e.g., BIP-340 signing keys in HSM, Jubjub encryption keys in TEE, contract private state in TEE sealed storage).

---

## Cryptographic Keys Per Signer

### Key Inventory

| Key | Type | Derivation | Secret? | Purpose |
|---|---|---|---|---|
| HD Seed | BIP39 mnemonic (24 words) | N/A | Yes | Source of all derived keys |
| `CoinPublicKey` | 32 bytes | SHA-256(domain + `CoinSecretKey`) | No | Identity via `ownPublicKey()`, used in contract-level authorization |
| `CoinSecretKey` | 32 bytes | SHA-256(domain + seed) | Yes | ZK witness -- proves coin ownership, nullifier computation (Zswap protocol) |
| `EncryptionPublicKey` | Jubjub point | `EncryptionSecretKey` * Generator | No | ECDH for shielded coin encryption; used as viewing key for indexer |
| `EncryptionSecretKey` | Jubjub scalar | Scalar sampling from seed with domain separator | Yes | Decrypt shielded coins, wallet proof witness (Phase 4b) |
| `DustSecretKey` | 32 bytes | Derived from seed | Yes | DUST fee payment, wallet proof witness (Phase 4b) |
| `SigningKey` | BIP-340 key (secp256k1) | BIP32 derivation from HD seed | Yes | Authorize unshielded token spends (Schnorr signature) |
| Contract-specific secrets | Varies (bytes, scalars, paths) | Varies | Yes | Contract-defined private state (authorization secrets, set membership proofs, etc.) |

### Key Usage by Phase

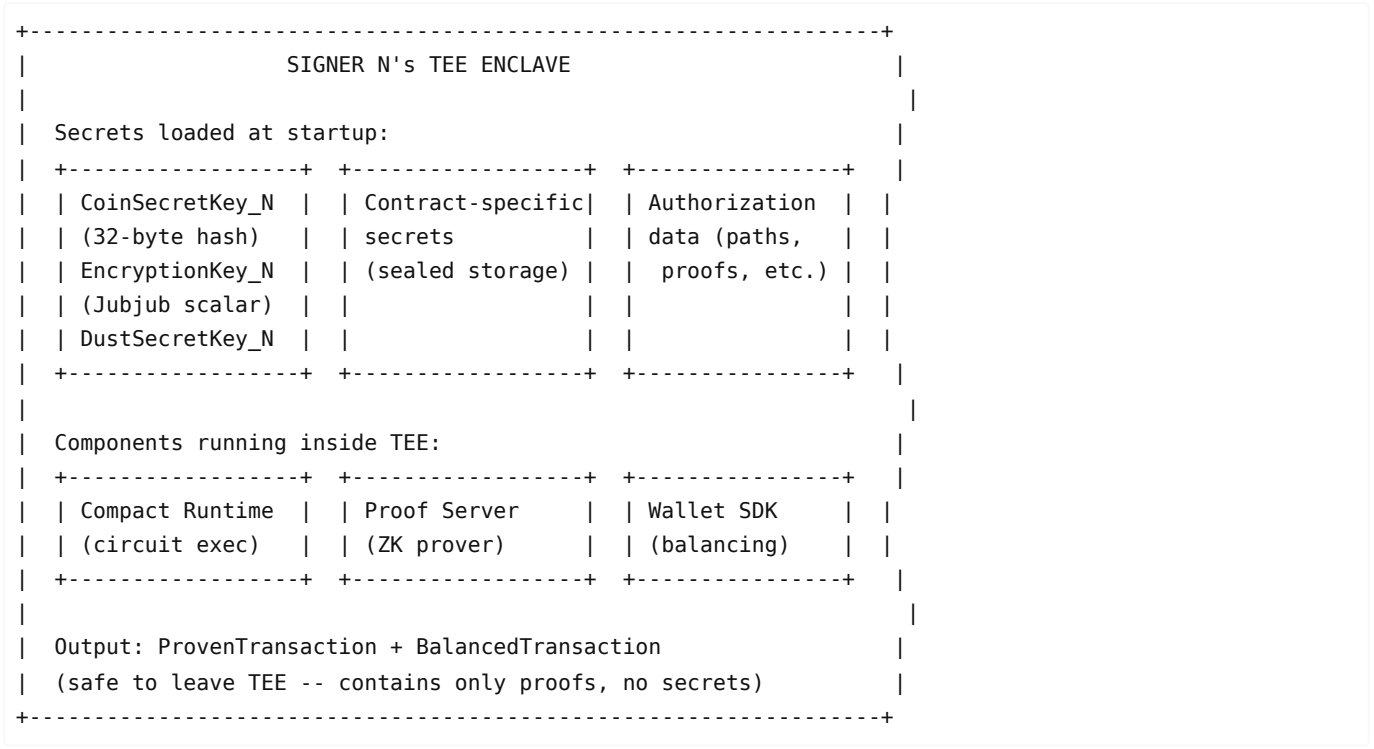
| Phase | `CoinPublicKey` | `CoinSecretKey` | `EncryptionSecretKey` | `DustSecretKey` | `SigningKey` | Contract secrets |
|---|---|---|---|---|---|---|
| 2. Circuit | Used | - | - | - | - | Used |
| 3. Contract Proof | - | Used | - | - | - | Used |
| 4. Balance | - | Used | Used | Used | - | - |
| 4b. Wallet Proof | - | Used | Used | Used | - | - |
| 5. Sign | - | - | - | - | If unshielded | - |

---

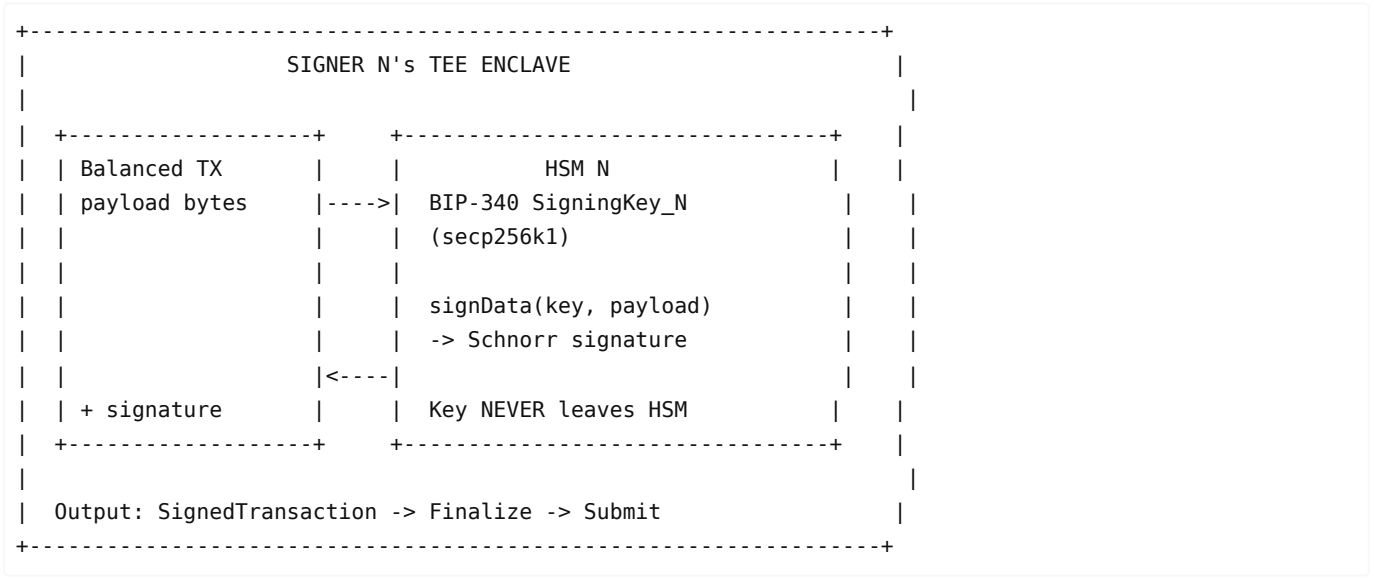
## Custodial Security Architecture

### Security Zones by Phase

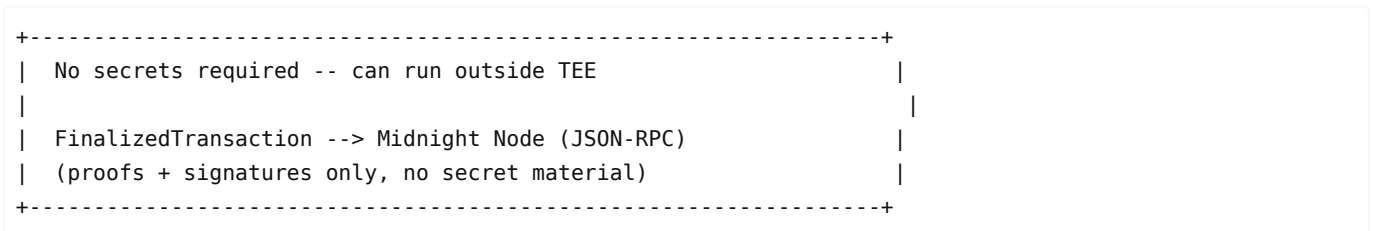
**Phases 1-4: All secrets in TEE**




**Phase 5: BIP-340 signing via HSM (optional)**

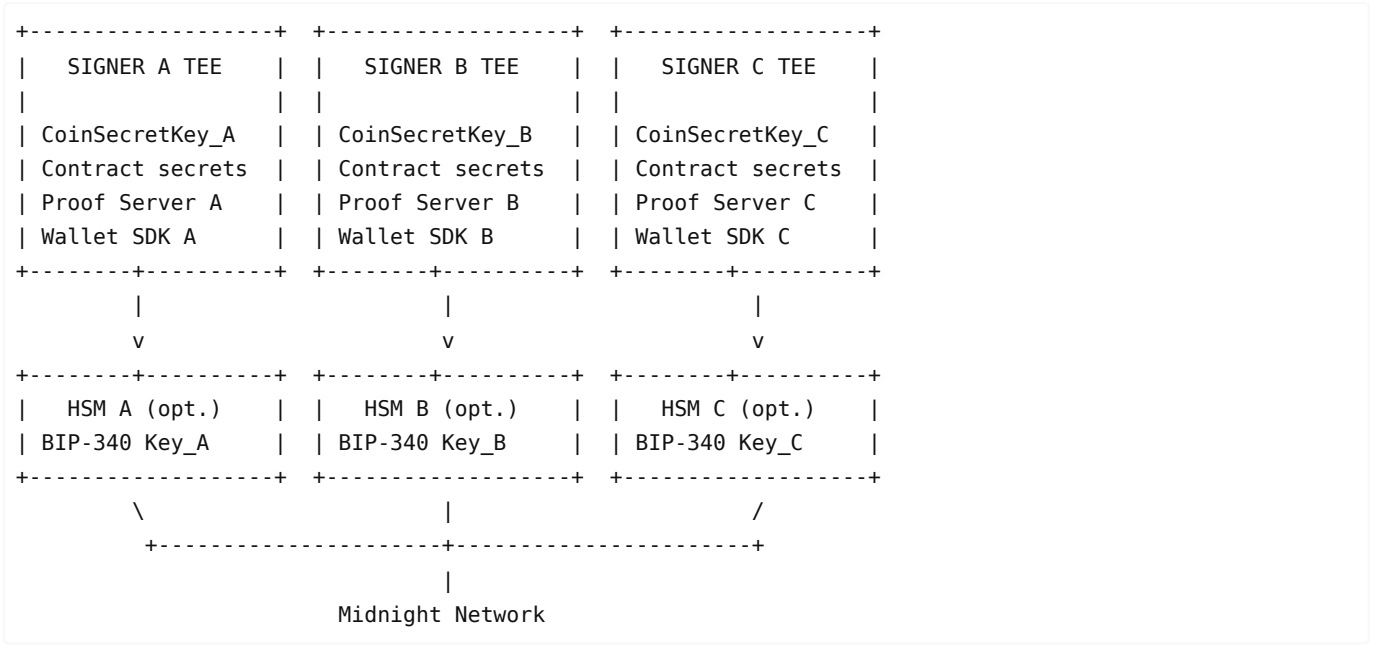



**Phases 6-9: No secrets needed**




## Independent TEE Per Signer

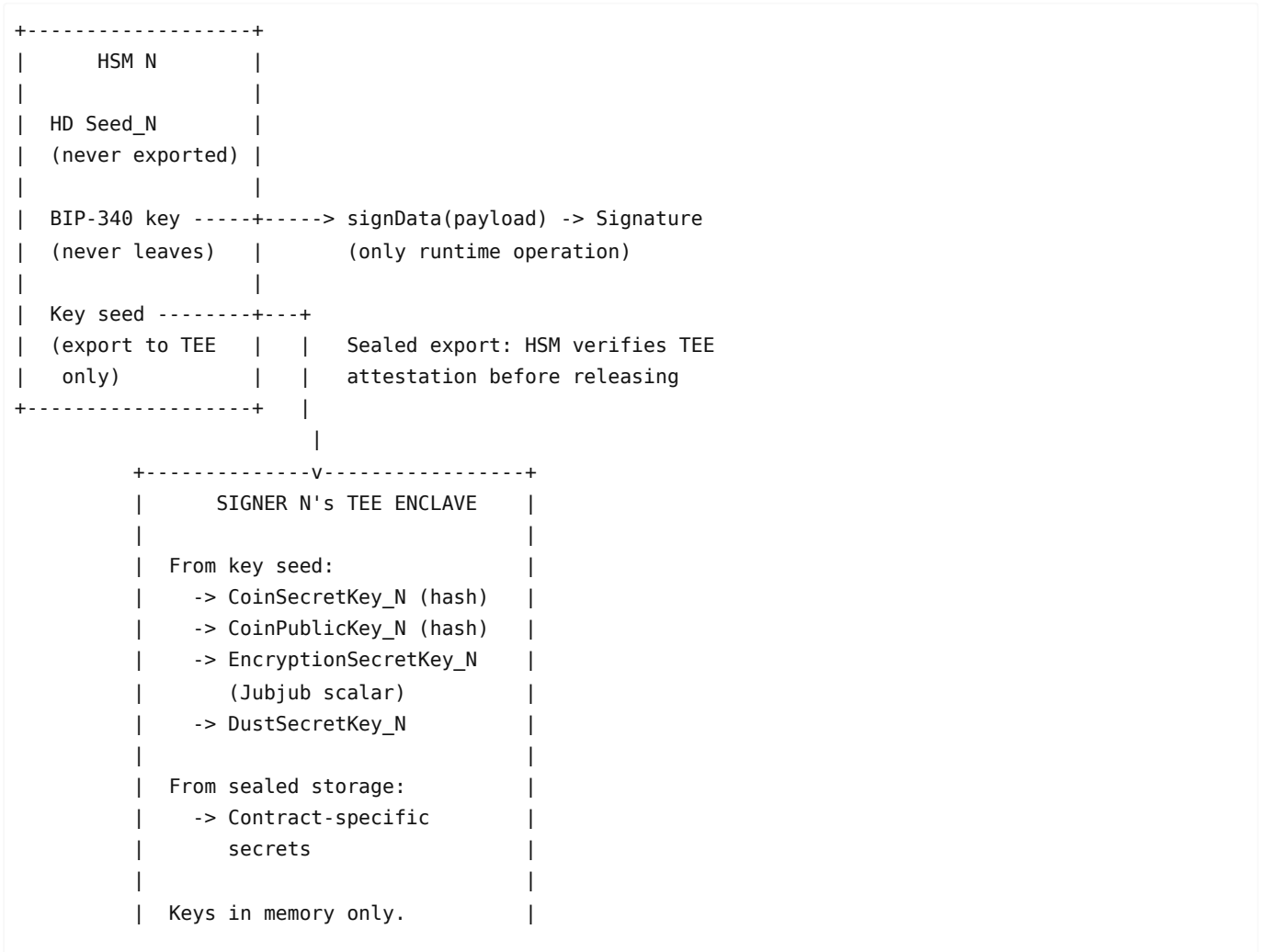
Each signer **must** have their own isolated TEE. A shared TEE defeats the purpose of multi-party authorization.




**Why independent TEEs:**

- Shared TEE = single point of failure; compromising it exposes all signer keys
- Independent TEEs mean an attacker must compromise M separate environments
- The M-of-N threshold only provides security if the M signers are independently secured

### Key Derivation Flow (Per Signer)



```
          |  Zeroed on shutdown.           |
          +--------------------------------+
```

### TEE vs HSM Decision

**TEE-only (sufficient for most custodial setups):**

```
Encrypted seed blob (cloud KMS, unique per signer)
         |
         v
+-------------------------------------------+
|           SIGNER N's TEE ENCLAVE          |
|                                            |
|  1. Decrypt seed (sealed to enclave ID)    |
|  2. Derive all keys (coin + encryption +   |
|     DUST + BIP-340)                        |
|  3. Load contract secrets (sealed storage) |
|  4. Execute circuit + generate ZK proof    |
|  5. Balance + sign + finalize              |
|  6. Submit TX                              |
|                                            |
|  All keys in memory only.                  |
|  Only finalized TX leaves.                 |
+-------------------------------------------+
```

**When to add HSM:**

| Scenario | TEE Only | TEE + HSM |
|---|---|---|
| Startup/small custodian | Sufficient | Over-engineered |
| Regulated financial institution | May not pass audit | Satisfies FIPS 140-2 Level 3 |
| Keys in same data center | Acceptable with strong TEE | Better -- HSM adds physical tamper resistance |
| Geographically distributed signers | Strong -- independent TEEs across regions | Marginal additional benefit |
| Budget-conscious | ~$135/signer/month | ~$1,535/signer/month |

**Recommendation:**

The M-of-N threshold is the primary security mechanism. Each layer adds defense-in-depth:

```
Layer 1: M-of-N threshold             Attacker needs M separate compromises
Layer 2: Geographic distribution       Attacker needs access to M facilities
Layer 3: TEE per signer               Attacker needs enclave exploit per signer
Layer 4: HSM per signer (optional)    Attacker needs physical HSM access per signer
```

For most deployments, Layers 1-3 provide sufficient security. Layer 4 (HSM) is primarily for regulatory compliance.

---

## Coordination Service

An off-chain service to notify signers of pending actions. Handles **no secrets**.

```
+-----------------------------------------+
|         Coordination Service            |
```

```
|         (stateless, no key material)        |
|                                              |
|  Responsibilities:                           |
|  - Watch indexer for contract state changes  |
|  - Notify signers via webhook / push / email|
|  - Serve dashboard UI for action status      |
|                                              |
|  Does NOT:                                   |
|  - Hold any keys                             |
|  - Sign any transactions                     |
|  - Access any TEE or HSM                     |
|  - Know which signer is which participant    |
+----------------------------------------------+
         |              |              |
    Webhook         Webhook         Webhook
         |              |              |
         v              v              v
    Signer A       Signer B       Signer C
```

### Notification Payloads

```
// New action requiring approvals
{
    event: 'action_created',
    contractAddress: '0xabc123...',
    actionId: 7,
    type: 'Transfer',
    currentApprovals: 0,
    requiredApprovals: 2,
    createdAtBlock: 154832,
}

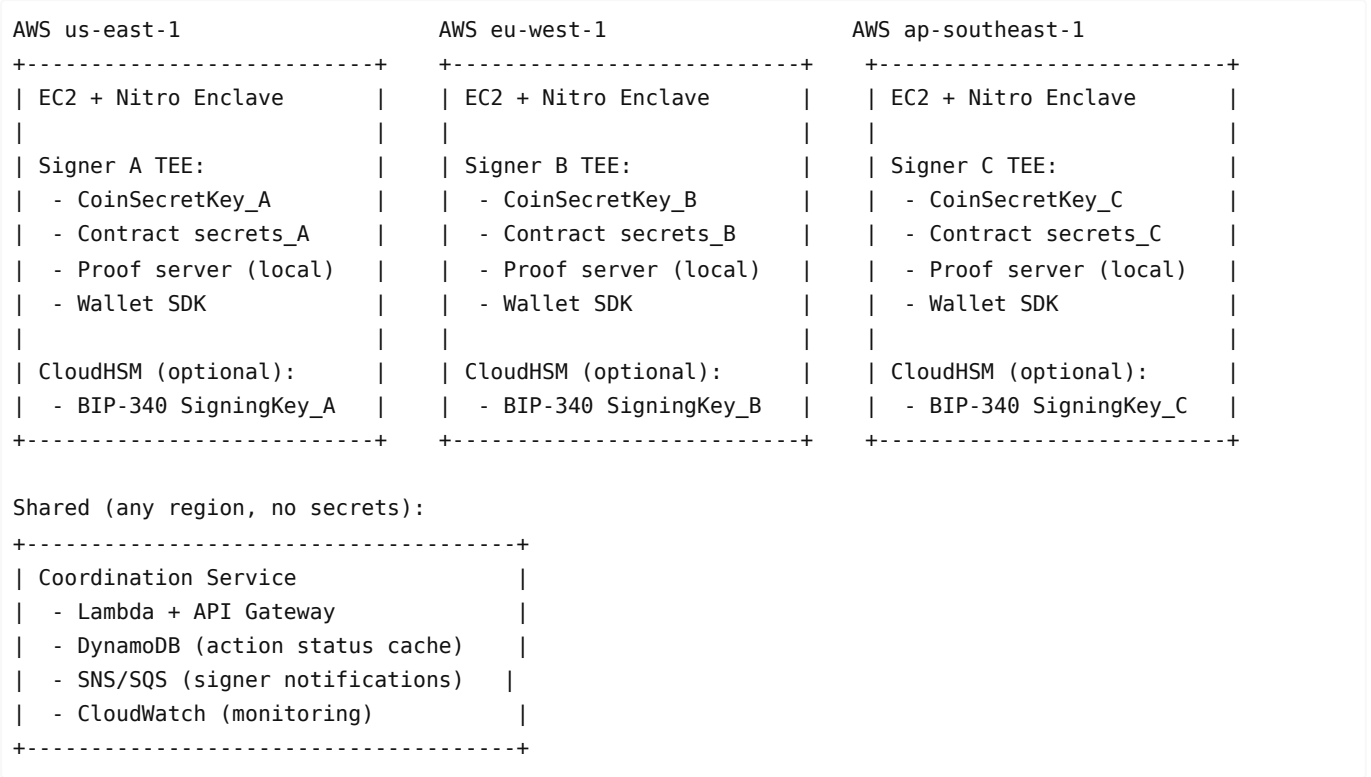
// Approval progress
{
    event: 'approval_added',
    contractAddress: '0xabc123...',
    actionId: 7,
    currentApprovals: 1,
    requiredApprovals: 2,
    thresholdMet: false,
}

// Threshold met -- ready for execution
{
    event: 'threshold_met',
    contractAddress: '0xabc123...',
    actionId: 7,
    currentApprovals: 2,
    requiredApprovals: 2,
    thresholdMet: true,
}
```

---

## Cloud Deployment

### AWS: M-of-N Multi-Party Wallet




### Cost Estimate (Per Signer)

| Component | AWS Service | Monthly Cost (approx.) |
|---|---|---|
| TEE | EC2 c5.xlarge + Nitro Enclave | ~$125 |
| HSM (optional) | CloudHSM cluster | ~$1,400 |
| Proof server | Runs inside TEE | Included in EC2 |
| Seed storage | KMS (encrypted blob in S3) | ~$1 |

| Configuration | Per Signer | 3 Signers Total |
|---|---|---|
| TEE only | ~$135/month | ~$405/month |
| TEE + HSM | ~$1,535/month | ~$4,605/month |
| Coordination service | -- | ~$15/month |

---

## Comparison: Single-Signer vs Multi-Party

| Aspect | Single Signer | Multi-Party (M-of-N) |
|---|---|---|
| Key compromise impact | Total loss | Partial -- attacker needs M signers |
| TEE instances needed | 1 | N (one per signer) |
| HSM instances needed | 0-1 | 0-N (optional per signer) |
| Proof server instances | 1 | N (one per signer TEE) |

| | | |
|---|---|---|
| Secrets in contract ZK proof | CoinSecretKey | CoinSecretKey + contract-specific secrets |
| Secrets in wallet ZK proof | CoinSecretKey + EncryptionSecretKey + DustSecretKey | Same (each signer has own wallet proof) |
| On-chain transactions per action | 1 | M+1 minimum (action + M approvals + execution) |
| Unshielded input signing | 1 BIP-340 per unshielded input | 1 BIP-340 per unshielded input per signer |
| Privacy of participants | N/A | Hidden via ZK proofs (contract-defined) |
| Recovery from single key loss | Must have seed backup | Other signers can vote to replace |
| Coordination overhead | None | Requires notification service |
| Fee cost per action | 1 tx fee | M+1 tx fees (each signer pays their own) |

---

## Secret Inventory and Backup Strategy

| Secret | Location | Backup | Recovery |
|---|---|---|---|
| HD Seed | HSM or TEE sealed storage | Shamir shares in geographically separate vaults | Reconstruct from shares |
| BIP-340 SigningKey | HSM (or TEE memory) | Re-derivable from HD seed | Re-derive in new TEE/HSM |
| CoinSecretKey (32-byte hash) | TEE memory only | Re-derivable from HD seed | Re-derive in new TEE |
| EncryptionSecretKey (Jubjub scalar) | TEE memory only | Re-derivable from HD seed | Re-derive in new TEE |
| DustSecretKey | TEE memory only | Re-derivable from HD seed | Re-derive in new TEE |
| Contract-specific secrets | TEE sealed storage | Encrypted backup in cold storage | Restore from backup; if lost, other signers must vote to re-register |
| Authorization data (e.g., paths) | TEE (recomputable) | Not needed -- can recompute from on-chain state | Query indexer and recompute |

### Disaster Recovery

**Single signer TEE failure:**

- Provision new TEE in same or different region
- Load HD seed from HSM or KMS backup
- Re-derive CoinSecretKey, EncryptionSecretKey, DustSecretKey, and SigningKey
- Restore contract secrets from encrypted cold storage backup
- Recompute authorization data from on-chain state

- Signer is operational again

**Single signer total loss (seed + secrets lost):**

- The lost signer can no longer prove authorization
- Remaining signers (if M-1 >= threshold for participant management) can:
  1. Propose removing the lost participant
  2. Propose adding a replacement participant (new identity)
  3. Approve and execute both actions
- Wallet continues operating with N-1 signers until replacement is added

**Multiple signer loss (fewer than M remaining):**

- If fewer than M signers remain operational, the wallet is **permanently locked**
- This is by design -- it prevents an attacker who compromises N-M+1 signers from draining funds
- Mitigation: use conservative thresholds (e.g., 2-of-3 rather than 3-of-3) and maintain robust backup procedures

---

*Last updated: 2026-03-04 Based on: Midnight SDK v7.0.0, Compact language version 0.21 Reference: TRANSACTION_LIFECYCLE.md for general lifecycle, MULTISIG_WALLET_LIFECYCLE.md for contract internals*